FFFA Donor Management: Handover Guide

28 October 2026 · Empact × FFFA AI Impact Sprint

This guide is for whoever takes the system forward for FFFA: a staff member using an AI assistant, a volunteer developer, or a freelancer. You don't need to be a programmer. An AI coding assistant can do the technical work if you give it clear instructions, and this guide gives you those instructions.

What you are receiving

Item	Where
The code	GitHub repository `FFFA-Donor-Management-System`
How to run it	`README.md` in the repository
How it's designed (database, rules, AI safeguards)	`docs/SYSTEM_DESIGN.md` (and `.pdf`)
How staff use it	`docs/staff-guide/STAFF_GUIDE.md` (and `.pdf`)
This guide	`docs/HANDOVER.md` (and `.pdf`)
Sample import files	`docs/samples/`

---

1. Take ownership of the code on GitHub

GitHub is where the code lives. FFFA should own its copy, so access never depends on a former volunteer.

1. **Create a GitHub account** for FFFA at github.com (use a shared FFFA email such as tech@ or admin@, not a personal one).
2. **Create a free organisation** for it: click your profile picture → *Your organizations* → *New organization* → *Free*. Name it, for example, `free-food-for-all`.
3. **Move the repository to FFFA.** The current owner opens the repository → *Settings* → *General* → scroll to *Danger Zone* → *Transfer ownership* → enter the FFFA organisation. You then accept the transfer from the FFFA account. After this, FFFA owns the code, the history and the documents.
4. **Give people access.** In the repository: *Settings* → *Collaborators and teams* → *Add people*. Use *Write* for people who make changes, and keep *Admin* for FFFA only. Remove access when someone leaves.
5. **Optional but recommended:** *Settings* → *Branches* → add a rule for `main` requiring a pull request before merging. This means every change gets looked at before it reaches the live system.

---

2. Set up a computer

You need one computer (Mac or Windows) with the following. All of them are free.

Install	From	Check it worked
Git	git-scm.com (Mac: it may already be installed)	Run `git --version` in Terminal
Node.js 20 or newer (LTS)	nodejs.org	Run `node --version`
GitHub Desktop (easiest way to sign in to GitHub)	desktop.github.com	Sign in with the FFFA GitHub account
An AI coding assistant that can read and change files and run commands, for example Claude Code, GitHub Copilot in VS Code, or Cursor	The provider's website	Open it and point it at a folder

Get the code onto the computer. In GitHub Desktop: *File* → *Clone repository* → pick `FFFA-Donor-Management-System` → choose a folder. Or ask your AI assistant:

> Clone https://github.com//FFFA-Donor-Management-System into my Documents folder.

---

3. Run it on your computer

Open the project folder in your AI assistant and say:

> Read README.md, then install and start this app with the demo data and open it in the browser.

Or type these yourself in a Terminal opened in the project folder:

```
npm install --legacy-peer-deps
npm run demo
```

Open **http://localhost:3000**. You are signed in automatically as a practice admin, with 9 made-up donors. Without an AI key, drafts use standard templates; section 6 covers adding one.

Command	What it does
`npm run demo`	Starts the app with the practice donors
`npm run dev`	Starts the app with an empty database
`npm test`	Runs the automatic checks (all should pass)
`npm run lint`	Checks the code for type errors
`rm -rf .data`	Wipes your local practice database (Windows: delete the `.data` folder)

Press **Ctrl + C** in the Terminal to stop the app.

---

4. Make changes safely with an AI assistant

Every change follows the same five steps. You can ask the assistant to do each one.

1. **Start a branch** (a separate copy for this change): *"Create a new branch called settings-screen from the latest main."*
2. **Describe the change** in plain words, and point the assistant at the design: *"Build X as described in docs/SYSTEM_DESIGN.md section 6. Follow the conventions in README.md."*
3. **Check it:** *"Run npm test and npm run lint and fix anything that fails. Then start the demo and show me the new screen."* Try it yourself in the browser.
4. **Save and share it:** *"Commit this with a clear message, push the branch and open a pull request."*
5. **Review and merge:** read the pull request on GitHub (or ask a second person to), then click *Merge pull request*.

Tips for good results

- Ask for one thing at a time. Small changes are easier to check.
- Always ask the assistant to run the tests before you merge.
- If it suggests something you don't understand, ask *"explain this in plain English and tell me the risks"* before saying yes.
- Never paste real donor data, passwords or API keys into the chat.
- The rules for donor messages (no guilt, never invent facts, major donors get a personal touch) are deliberate. Keep them unless FFFA decides otherwise.

---

5. What the system does today, and what it doesn't yet

Built and tested (each has automatic tests):

- Donor records: individuals, companies and grant funders with a contact person, gifts (money and goods), notes, conversations, key dates, archive, and PDPA delete (admin only)
- Follow-up rules (thank-you, failed payment, key date, going cold, lapsed, anniversary, upgrade ask) and the follow-up queue
- AI-drafted messages: draft → edit → approve → mark as sent, with major donors blocked and contact details never sent to the AI
- Giving.sg / GIVE.asia CSV import with matching of the same person across platforms
- Dashboard, activity log, admin settings (rules, tiers, S$10 = 1 meal)

Not built yet. The **Phase 1 status** table in `README.md` is always the up-to-date list. At handover, these were outstanding:

Item	Needed to go live?	What to ask the AI assistant
Real login (Supabase Auth). Right now the app only runs in practice mode on a laptop, and it refuses to start on a server without login.	**Yes**	*"Implement supabaseStaffResolver in server/middleware/auth.ts using Supabase Auth, as described in docs/SYSTEM_DESIGN.md. Only emails in the staff table may sign in. Add a login screen."*
Daily automatic run of the follow-up rules. Today the rules run after an import and when `POST /api/follow-ups/refresh` is called.	Recommended	*"Run the rules engine (runRulesEngine) once a day at 7am Singapore time on the server."*
Screens still missing from the list in the design (check the README table)	Depends	*"Build the screen using the existing API in server/routes, matching the style of the drafting screen."*

---

6. Put it online

This is a one-time setup, best done with the AI assistant. Each step names the website and what to copy where. Keep all keys secret: they go in the hosting settings, never in the code or in chat.

Step 1: Database and login (Supabase). Create a free project at supabase.com, in the Singapore region. Copy the **connection string** (*Project Settings → Database*). It becomes `DATABASE_URL`. The app creates all its tables automatically the first time it starts. Turn on email login under *Authentication*.

Step 2: Finish the login. Use the prompt in section 5. Then add each staff member to the `staff` table with their email and role (`staff` or `admin`).

Step 3: AI key (Google Gemini). At aistudio.google.com create an API key, and **turn on billing for it**. On the paid tier, Google does not use FFFA's data to improve its products. That matters for donor information. The key becomes `GEMINI_API_KEY`. Without it, the app still works and uses templates.

Step 4: Hosting (for example Render). At render.com create a *Web Service* from the FFFA GitHub repository:

Setting	Value
Build command	`npm install --legacy-peer-deps && npm run build`
Start command	`npm start`
Environment variables	`DATABASE_URL`, `GEMINI_API_KEY`, `AUTH_MODE=supabase`, plus the Supabase keys the login code needs

Render gives you a web link. Every merge to `main` updates the live site automatically. Any host that runs Node.js works the same way (for example Google Cloud Run or Railway). Ask the assistant: *"Help me deploy this to Render. Walk me through each screen."*

Step 5: Before entering real donors.

- [ ] Login works, and someone not in the staff table cannot get in
- [ ] `DATABASE_URL` points to Supabase (without it the app falls back to a temporary database that is wiped when the server restarts)
- [ ] Supabase daily backups are on
- [ ] Practice donors are not in the live database (never run `npm run demo` against it)
- [ ] One test donor goes through the full flow: add → follow-up → draft → approve → mark as sent

---

7. Data protection (PDPA)

- Donor data lives only in FFFA's Supabase database. The AI receives donor names, giving history and notes with phone numbers, emails and ID numbers removed. It never receives contact details.
- No NRIC, FIN, UEN or bank details are stored. If this is needed later, the design (section 3) explains how to add it in a separate, admin-only, encrypted table.
- Admins can permanently delete a donor on request (PDPA). Every change is recorded in the activity log.
- The `.env` file (where local keys go) is never uploaded to GitHub. Keep it that way.

---

8. If something goes wrong

Problem	Fix
`npm install` fails with an "ERESOLVE" or "edgesOut" error	Use `npm install --legacy-peer-deps`
"Port 3000 is already in use"	Another copy is running. Close it, or run `PORT=3001 npm run demo`
Practice data looks wrong or duplicated	Stop the app, delete the `.data` folder, run `npm run demo` again
Tests fail after a change	Ask the assistant: *"npm test fails. Find out why and fix it without deleting tests."*
Live site errors	Hosting dashboard → *Logs*. Paste the error into your AI assistant (remove any keys or donor details first).
Drafts always say "Template"	`GEMINI_API_KEY` is missing or has no billing. Check the hosting environment variables.

---

9. Hiring help

To bring in a developer or freelancer, add them on GitHub with *Write* access (section 1), share this guide, `README.md` and `docs/SYSTEM_DESIGN.md`, and ask for every change as a pull request that you approve. Remove their access when the work is done. Keep the Supabase, Google and hosting accounts in FFFA's name.